

Disaster Alert & Community Response App

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Lankey Tejasri :2520090017

Dikshita Siripireddy :2520090007

Veeramallu Rohita :2520090235

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Declaration

The Project Report entitled **“Disaster Alert & Community App”** is a record of Bonafide work of **Lankey Tejasri -2520090017 , Dikshita Siripireddy - 2520090007 , Veeramallu Rohita – 2520090235** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Lankey Tejasri –2520090017
Dikshita Siripireddy –2520090007
Veeramallu Rohita–2520090253

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

CERTIFICATE

This is certify that the project report entitled **“Disaster Alert & Community App”** is a bonafide work done and submitted by **Lankey Tejasri - 2520090017 , Dikshita Siripireddy- 2520090007, Veeramallu Rohita - 2520090235** in partial fulfillment of the of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 1, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr**, and my guide **Dr Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the ***Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Lankey Tejasri – 2520090017

Dikshita Siripireddy – 2520090007

Veeramallu Rohita– 2520090235

Abstract

The **Disaster Alert & Community Response App** is a centralized platform designed to improve communication and coordination during disasters such as floods, cyclones, earthquakes, fires, and other emergencies. During such situations, people may face difficulties in receiving timely alerts, reporting incidents, finding nearby assistance, and communicating with response teams. The proposed application provides a digital platform where users can receive disaster alerts, report emergency incidents, request assistance, and offer help to affected people. The system maintains user, disaster, incident, and response information using suitable database technologies. A distributed backend architecture with REST APIs enables efficient communication between the application and backend services. Authentication and authorization can be implemented to protect user and emergency-related information. The system can use separate services for users, disaster alerts, incident management, and community responses, making the application scalable and maintainable. The application also supports **location-based incident reporting and map-based disaster tracking**, allowing users and response teams to identify affected areas and locate reported incidents. Users can request different types of emergency assistance, such as **medical support, rescue, food, water, shelter, and transportation**. Community members can register as volunteers and offer assistance based on their availability and location. The platform can also provide information about **nearby shelters, hospitals, police stations, and other emergency services** to help users find appropriate support during critical situations.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	7
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	8
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	9
5	Design	5.2 ER Diagram / Database schema	10
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	12
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	13
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	15
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	17
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	18
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	19
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	20

13	References	- Books, tutorials, APIs, documentation sites used	21
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Natural disasters such as floods, cyclones, earthquakes, fires, landslides, and other emergencies can affect people within a very short period of time. During such situations, people often face difficulties in receiving timely information, reporting incidents, requesting emergency assistance, and finding nearby shelters or emergency services.

The Disaster Alert & Community Response App is designed as a centralized digital platform that connects affected users, community volunteers, and emergency response teams. Instead of depending on scattered communication channels, the application provides a single platform for disaster alerts, incident reporting, emergency help requests, community assistance, and location-based emergency information.

The proposed system allows users to receive disaster alerts, report incidents with their location, request different types of emergency assistance, and offer help to affected people. The system also provides map-based information about incidents, shelters, hospitals, police stations, and other emergency services.

The project aims to improve communication, information sharing, and coordination during emergency situations by bringing important disaster-response services together in one platform. The original project proposal also identifies timely alerts, location-based incident reporting, emergency assistance, community participation, and map-based tracking as the main objectives of the system.

1.2 Problem Statement

During disasters, information may be delayed, scattered across different communication channels, or difficult to verify. Affected people may not know where to request help, while volunteers and response teams may not know where assistance is required.

The system therefore needs to address the following questions:

1. How can users receive timely disaster alerts?
2. How can users report an emergency incident quickly?
3. How can emergency help requests be communicated to nearby responders?
4. How can volunteers identify people who need assistance?
5. How can users locate nearby shelters and emergency services?

6. How can disaster incidents and alerts be represented on a map?
7. How can user and emergency information be managed securely?

The proposed application addresses these problems by providing a centralized platform for disaster alerts, incident reporting, help requests, community response, and emergency-location services.

1.3 Scope of the Project

The system covers the major activities required during a disaster-response situation.

The scope includes:

- **User registration and login**
- **User profile management**
- **Emergency contact management**
- **Disaster alerts**
- **Incident reporting**
- **Location-based incident reporting**
- **Emergency SOS**
- **Emergency help requests**
- **Priority-based help requests**
- **Community/volunteer assistance**
- **Live disaster map**
- **Nearby shelters**
- **Hospitals and medical services**
- **Police and emergency services**
- **Official announcements**
- **Disaster information and safety instructions**
- **Response-team/community coordination**
- **Storage and management of disaster-related information**

The system focuses on providing a centralized web-based platform. Mobile applications, advanced hardware-based disaster detection, satellite-based monitoring, and direct integration with government emergency systems can be considered future enhancements.

1.4 Objectives of the Project

The main objectives of the project are:

- **To provide timely disaster alerts to users.**
- **To allow users to report emergency incidents.**
- **To capture the location of reported incidents.**
- **To provide an emergency SOS mechanism.**
- **To allow users to request emergency assistance.**
- **To classify help requests according to priority.**
- **To allow community members to offer assistance.**
- **To connect nearby volunteers with people requiring help.**
- **To provide information about nearby shelters.**
- **To help users locate hospitals, police stations, and emergency services.**
- **To display disaster-related incidents on a map.**
- **To provide official announcements and updates.**
- **To securely manage users and emergency-related information.**
- **To improve communication and coordination during disasters.**

These objectives are consistent with the project's proposed goals of alerts, incident reporting, emergency assistance, community help, nearby services, official updates, and map-based disaster tracking.

1.5 Importance of the Application

The application is important because disaster situations require quick access to reliable information and assistance.

The system can:

- **Reduce delays in reporting incidents.**
- **Help users quickly communicate their emergency needs.**
- **Improve visibility of affected locations.**
- **Help volunteers identify assistance requests.**
- **Provide information about nearby safe locations.**
- **Centralize disaster alerts and announcements.**
- **Improve coordination between affected people and responders.**
- **Provide location-based emergency information.**

The centralized design combines alerts, reports, help requests, and community assistance into one platform.

1.6 Target Users / Audience

The application can support multiple categories of users.

1. Affected Users

Users who require emergency assistance can:

- Receive alerts
- Report incidents
- Send SOS requests
- Request help
- Share their location
- Find shelters and emergency services

2. Community Volunteers

Volunteers can:

- View assistance requirements
- Offer help
- Provide food/water
- Assist with transportation
- Provide other permitted support
- Respond to nearby help requests

3. Emergency Response Teams

Response teams can:

- Monitor incidents
- View help requests
- Prioritize emergencies
- Track locations
- Coordinate responses

4. Administrators

Administrators can:

- Manage users
- Manage alerts
- Manage incidents
- Manage shelters
- Manage emergency services
- Monitor help requests
- Manage official announcements

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has relatively modest hardware requirements for development and demonstration.

Minimum Requirements

- Dual-core processor
- 4 GB RAM
- 10 GB available storage
- Internet connection
- Keyboard and mouse
- Monitor/display

Recommended Requirements

- Intel Core i5 / equivalent processor
- 8 GB RAM or higher
- SSD storage
- Stable internet connection

2.2 Software Requirements

The system can be developed using:

- Windows/Linux
- Node.js
- npm
- React.js
- Vite
- Python
- FastAPI
- PostgreSQL

- Visual Studio Code
- Git
- GitHub
- Postman
- Modern web browser
- Leaflet
- OpenStreetMap

2.3 Development Tools and Frameworks

Frontend

- React.js
- Vite
- React Router
- CSS
- Axios

Backend

- FastAPI
- Python
- REST APIs
- Pydantic

Database

- PostgreSQL

Mapping

- Leaflet

- OpenStreetMap

Testing

- Postman
- Browser testing

Version Control

- Git
- GitHub

The original project proposal describes REST APIs, authentication, database storage, and modular backend services as core parts of the intended architecture.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The frontend is developed using React.js with Vite.

React provides reusable components for:

- Navigation
- Alert cards
- Emergency buttons
- Forms
- Dashboard cards
- Maps
- Shelter information
- Help requests

React Router is used for navigation between pages such as:

- Home

- Login
- Register
- Dashboard
- Alerts
- Incident Report
- Request Help
- Offer Help
- Live Map
- Shelters
- Profile

Axios can be used to communicate with the backend REST APIs.

3.2 Back-End Design

The backend is implemented using FastAPI.

FastAPI provides REST APIs for:

- Authentication
- Users
- Disaster alerts
- Incident reports
- Help requests
- Community responses
- Shelters
- Emergency services

The backend handles validation, authentication, authorization, business logic, and communication with the database.

3.3 Database Design

PostgreSQL is used for persistent storage.

Possible tables include:

1. Users
2. Emergency Contacts
3. Disaster Alerts
4. Incident Reports
5. Help Requests
6. Community Responses
7. Shelters
8. Emergency Services
9. Announcements
10. Notifications

The project's original design specifically identifies users, alerts, incidents, emergency/help requests, community responses, and shelter/emergency-service information as major data components.

3.4 Version Control

Git and GitHub are used for:

- Source-code management
- Version control
- Collaboration
- Project backup

- Tracking changes

3.5 Authentication and Security

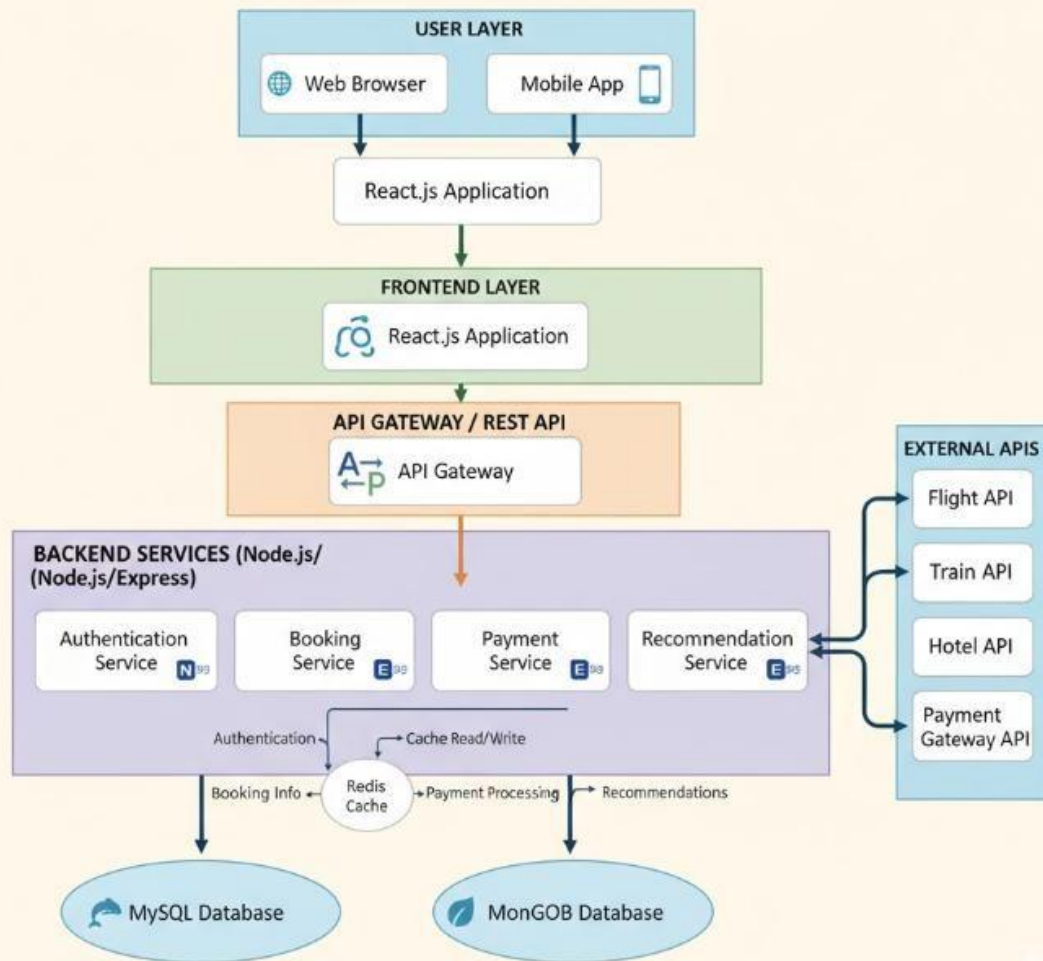
The application can use:

- JWT authentication
- Password hashing
- Role-based authorization
- Secure API endpoints
- Input validation
- Protected user information

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram

Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations



4.2 Description of Each Layer (Frontend, Backend, Database)

The presentation layer is the React frontend.

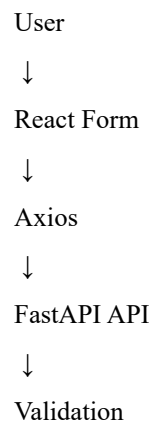
It provides:

- Login
- Registration
- Dashboard
- Alerts
- Incident reporting
- SOS
- Help requests
- Volunteer assistance
- Live map
- Shelters
- Profile

The frontend communicates with the backend through REST APIs rather than directly accessing the database.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The FastAPI backend processes requests from the frontend.

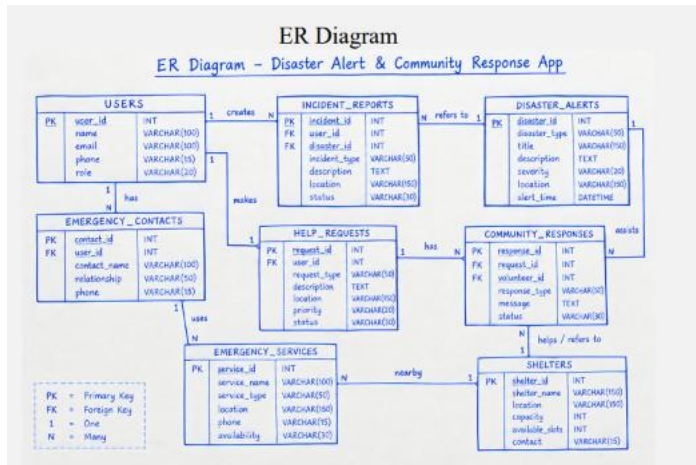


↓
Database

↓
Response

↓
React UI

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The application is divided into the following modules:

Module 1 — Authentication

Provides:

- Registration
- Login
- Logout
- JWT authentication
- Role management

Module 2 — Disaster Alerts

Provides:

- Disaster notifications
- Severity
- Location
- Alert description
- Official updates

Module 3 — Incident Reporting

Allows users to:

- Select disaster type
- Select severity

- Enter title
- Enter description
- Enter location
- Share GPS location
- Submit incident

Module 4 — Emergency Help

Allows users to request:

- Medical assistance
- Rescue
- Food/water
- Shelter
- Transportation
- Other assistance

Module 5 — Community Assistance

Allows volunteers to:

- Register availability
- Select assistance type
- Share location
- Provide contact details
- Respond to help requests

Module 6 — Live Map

Displays:

- Reported incidents
- Disaster locations
- Shelters
- Hospitals
- Police stations
- Emergency services

Module 7 — Shelter and Emergency Services

Provides information about:

- Safe shelters
- Hospitals
- Police stations
- Emergency service locations

Module 8 — User Profile

Allows users to manage:

- Name
- Email
- Phone
- Emergency contacts
- Location
- Account information

Section 6.2 – Front-End Logic (UI Rendering and State Management)

React components are used to create reusable UI elements.

Examples:

Navbar

AlertCard

EmergencyButton

QuickAction

MapMarker

ShelterCard

HelpRequestCard

React state manages:

- Form data
- Alerts
- Filters
- Login state
- Help requests
- Map information

6.3 API Integration

All HTTP calls go through one central Axios instance, which automatically attaches the logged-in user's JWT to every outgoing request and automatically logs the user out if any response comes back with a 401 Unauthorized status (for example, after the token expires). Each backend resource has a matching frontend “service” file (roomService.js, eventService.js, and so on) so that no page ever hardcodes a URL or repeats the same fetch logic.

6.4 Authentication and Authorisation

Registration always creates a student account — there is deliberately no way to create an admin account through the public API, so an attacker (or a curious student) cannot register their way into administrator access. Admin accounts exist only because they were inserted directly into the database. Every protected route runs a `protect` middleware that verifies the JWT, and admin-only routes additionally run an `adminOnly` middleware that checks the token's role claim, rejecting any student token with a 403 Forbidden response.

CHAPTER 7- FEATURES

7.1 List of Main Features

User Features

- Register
- Login
- Manage profile
- Add emergency contacts
- Receive alerts
- Report incidents
- Request emergency help
- Send SOS
- View live disaster map
- Find shelters
- Find emergency services

Volunteer Features

- Offer assistance
- Select availability
- Select assistance type
- View suitable help requests
- Respond to requests
- Update response status

Administrator Features

- Manage users
- Create disaster alerts

- Manage incident reports
- Manage help requests
- Manage shelters
- Manage emergency services
- Publish announcements
- Monitor disaster activity

7.2 How Each Feature Works

The Disaster Alert & Community Response App provides multiple features that help users during emergency situations. The main user features include **User Registration and Login, Emergency SOS, Disaster Alerts, Incident Reporting, Emergency Help Requests, Offer Help, Live Disaster Map, Nearby Shelters, Emergency Services, and Profile Management.**

The **Emergency SOS** feature allows users to quickly request urgent assistance during a critical situation. When the SOS option is activated, the system can send an emergency request containing the user's location and basic user information to the response system. This helps response teams identify the user's location and provide assistance as quickly as possible.

The **Disaster Alerts** feature provides users with important information about emergencies such as floods, heavy rainfall, fires, cyclones, and other disasters. Alerts can include the disaster type, severity, affected location, description, and time.

The **Incident Reporting** feature allows users to report disasters or dangerous situations by entering the incident type, severity, location, title, and description. This information can help response teams understand the situation and take appropriate action.

The **Emergency Help Request** feature allows users to request specific assistance such as medical help, rescue, food, water, shelter, or transportation. Users can provide their location, number of people requiring help, contact number, priority, and description of the situation.

The **Offer Help** feature allows community members to volunteer during emergencies. Users can specify the type of assistance they can provide, their availability, location, contact number, and relevant experience.

The **Live Disaster Map** displays emergency-related locations such as reported incidents, disaster alerts, shelters, and emergency services. This helps users understand affected areas and locate available assistance.

The **Shelter and Emergency Services** feature helps users find nearby safe shelters, hospitals, police stations, and other emergency facilities.

The **Profile Management** feature allows users to manage their personal information and emergency contact details. Together, these features provide users with a centralized platform for receiving information, requesting assistance, reporting emergencies, and supporting other people during disasters.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman is used to test the communication between the React + Vite frontend and the FastAPI backend through REST APIs. It helps verify whether the backend correctly receives requests, processes the data, communicates with the PostgreSQL database, and returns the expected response to the frontend.

The main APIs of the Disaster Alert & Community Response App can be tested using different HTTP methods such as GET, POST, PUT, and DELETE. For example, the registration API can be tested using a POST request by sending user details such as name, email, phone number, and password. Similarly, the login API can be tested to verify user authentication. The incident reporting and emergency help request APIs can be tested by sending incident type, severity, location, description, priority, and other required information.

Postman testing can also be used to verify disaster alerts, SOS requests, community assistance, shelters, emergency services, and user profiles. The response status codes and returned JSON data are checked to ensure that the APIs are working correctly.

8.2 Integration Testing

Integration testing is performed to verify that the different components of the Disaster Alert & Community Response App work correctly together. The React + Vite frontend is integrated with the FastAPI backend through REST APIs, while the backend communicates with the PostgreSQL database to store and retrieve application data.

The integration testing process verifies important workflows such as user registration and login, disaster alert retrieval, incident reporting, SOS requests, emergency help requests, community assistance, shelter information, and emergency service information. For example, when a user submits an incident report through the frontend, the data should be sent to the FastAPI backend, validated and stored in the database, and the appropriate response should be returned to the frontend.

The testing also checks whether location information, severity levels, priority values, user information, and emergency requests are correctly transferred between the frontend, backend, and database. This ensures that the major modules of the application communicate properly and that data is not lost or incorrectly processed during the interaction between components.

8.3 Tools Used for Testing

The Disaster Alert & Community Response App uses different tools to test the functionality and interaction between its frontend, backend, and database components. Postman is used to test REST APIs by sending GET, POST, PUT, and DELETE requests and checking the responses returned by the FastAPI backend. React + Vite is used to test the frontend pages and user interactions such as login, registration, incident reporting, SOS, help requests, and navigation between modules. FastAPI provides the backend API environment for testing authentication, alerts, incidents, help requests, and community response services. PostgreSQL is used to verify whether user, alert, incident, help request, shelter, and emergency service data is correctly stored and retrieved. Web browser developer tools can also be used to identify frontend errors, API request failures, and JavaScript issues during application testing.

8.4 Test Cases and Results

The Disaster Alert & Community Response App was tested using different test cases to verify the functionality of its major modules. The testing covered user registration and login, form validation, disaster alerts and alert filtering, incident reporting, emergency help requests, SOS functionality, community assistance, live map, shelters, and navigation between different pages. The test cases verified whether valid information was accepted correctly, required fields displayed appropriate validation messages when left empty, and users were redirected to the correct pages after completing an action. The alert filtering functionality was also tested to ensure that selecting options such as Critical, High Priority, Weather, and Flood displayed the corresponding alerts. These tests helped verify that the frontend modules and user interactions were functioning as expected. Backend and PostgreSQL integration testing will be performed after the FastAPI backend and database are connected to the frontend.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the Disaster Alert & Community Response App involves setting up the frontend, backend, and database so that the complete application can be accessed and used in a production environment. First, the React + Vite frontend is prepared by installing the required dependencies and creating a production build using the appropriate build command. The generated frontend files can then be hosted on a suitable web server or deployment platform.

Next, the FastAPI backend is configured with the required environment variables, dependencies, and database connection details. The backend is started using a production-ready server configuration so that the REST APIs can receive requests from the deployed frontend. The PostgreSQL database is then configured and connected to the FastAPI backend, and the required tables for users, disaster alerts, incidents, help requests, community responses, shelters, and emergency services are created.

After deploying the backend and database, the frontend API configuration is updated with the deployed backend URL instead of the local development URL. The complete application is then tested to verify frontend navigation, API communication, authentication, incident reporting, SOS requests, help requests, alerts, and database operations. Finally, application logs, error handling, database connectivity, and security configurations are checked to ensure that the deployed system operates correctly.

9.2 Environment Configuration

The application environment is configured to support the React + Vite frontend, FastAPI backend, and PostgreSQL database. The frontend development environment uses **Node.js and npm** to install dependencies and run the React application. The backend environment uses **Python and FastAPI**, with the required Python packages installed for API development and database communication. PostgreSQL is configured as the database environment for storing users, disaster alerts, incident reports, help requests, community responses, shelters, and emergency service information. Environment variables can be used to store configuration details such as the database connection URL, API settings, and authentication-related information instead of placing sensitive values directly in the source

code. During development, the frontend can run on the Vite development server and communicate with the FastAPI backend through its configured API address.

9.3 Hosting of Frontend and Backend

The **frontend** of the Disaster Alert & Community Response App is developed using **React + Vite** and can be hosted on a web hosting platform or web server after creating the production build. The frontend provides the user interface for features such as disaster alerts, incident reporting, SOS, help requests, community assistance, and map-based services.

The **backend** is developed using **FastAPI** and provides REST APIs for communication between the frontend and the database. It can be hosted on a server or cloud platform where the FastAPI application runs continuously and handles requests from the deployed frontend. The backend is connected to the **PostgreSQL database**, which stores user, alert, incident, help request, community response, shelter, and emergency service information. After both components are hosted, the frontend is configured with the backend API address so that users can access the complete application through the deployed web interface.

performance insights, feeding into long-term optimisation cycles.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of the Disaster Alert & Community Response App, several challenges were encountered while designing and integrating the different modules. One of the main challenges was creating a clear and user-friendly interface for emergency situations, where important features such as SOS, disaster alerts, incident reporting, and help requests need to be easily accessible. Another challenge was implementing proper navigation between the different React pages and ensuring that forms performed the required validation.

The project also involved challenges in planning the communication between the React + Vite frontend and FastAPI backend through REST APIs. Managing user authentication, incident information, emergency help requests, and other application data required careful API and database design. Location-based features such as the live disaster map, shelters, and emergency services also required additional consideration for handling geographical information. Furthermore, integrating the planned PostgreSQL database with the backend and ensuring that information is correctly stored and retrieved required additional testing. These challenges were addressed through step-by-step development, frontend testing, API testing, and integration testing.

10.2 Solutions Applied

To overcome the issues faced during development, the application was developed using a modular approach, with separate React pages and components for major functions such as login, disaster alerts, incident reporting, SOS, help requests, community assistance, maps, and shelters. Form validation was added to ensure that required information is entered correctly before submission. React Router was used to provide proper navigation between application modules.

The application architecture was planned with React + Vite for the frontend, FastAPI for the backend, and PostgreSQL for data storage. REST APIs were planned to establish communication between the frontend and backend. Testing with Postman was considered for verifying API requests and responses, while integration testing was used to check communication between the frontend, backend, and database. Location-based features were planned using map integration so that incidents, shelters, and emergency services can be represented according to their locations. These approaches helped organize the development process and improve the reliability and maintainability of the application.

10.3 Current Limitations or Known Bugs

The current version of the Disaster Alert & Community Response App has some limitations because the complete backend and database integration are still under development. The **login and registration modules currently use frontend validation and are not yet connected to real user authentication**. Similarly, incident reports, emergency help requests, and community assistance submissions are currently handled on the frontend and are not yet permanently stored in the PostgreSQL database.

The **SOS feature and GPS location detection** require backend and location-service integration to provide real-time emergency communication. The live disaster map, shelters, and emergency services also require actual location data and backend APIs for dynamic information. Some dashboard values and disaster alerts currently use sample data rather than live database information. Real-time notifications, automatic volunteer matching, and continuous communication with emergency response teams are also not fully implemented. These limitations are planned to be addressed during further development and backend integration.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The future development of the Disaster Alert & Community Response App will focus on improving its emergency response capabilities and making the system more useful in real-world disaster situations. Planned features include real-time disaster alerts and notifications, allowing users to receive important emergency information quickly. The system can also be enhanced with real-time location tracking and improved map-based disaster monitoring to identify affected areas and nearby assistance.

The application can further support automatic matching of volunteers with emergency help requests based on their location, availability, and type of assistance. Additional features such as offline or low-network support, multilingual communication, mobile application support, AI-based disaster detection and risk prediction, and integration with government or emergency response services can also be implemented. These enhancements are intended to improve communication, coordination, and the overall effectiveness of community-based disaster response.

11.2 Possible Integrations or Optimisations

The Disaster Alert & Community Response App can be further improved by integrating additional services and optimizing the existing system. **Map and location services** can be integrated to provide accurate incident locations, nearby shelters, hospitals, police stations, and other emergency services. **Real-time notification services** can be integrated to deliver important disaster alerts and emergency updates to users quickly.

The system can also be optimized by improving **API response time, database queries, data validation, and frontend performance**. Caching can be used for frequently accessed information such as disaster alerts and emergency service locations. **WebSocket-based communication** can be considered for real-time updates between users and response teams. Additional integrations such as **SMS or email notifications, cloud storage for emergency evidence, multilingual support, and AI-based disaster analysis** can further improve the functionality of the application.

Chapter -12 Conclusion

12.1 Summary of the Project

The Disaster Alert & Community Response App is a centralized platform designed to improve communication, coordination, and assistance during disasters and emergency situations. The application allows users to receive disaster alerts, report incidents, request emergency help, activate SOS, offer assistance to affected people, and access nearby shelters and emergency services.

The system is designed using a React + Vite frontend, FastAPI backend, and PostgreSQL database. REST APIs are used to enable communication between the frontend and backend, while the database stores information related to users, disaster alerts, incidents, help requests, community responses, shelters, and emergency services. Location-based features and map integration support the identification of incidents and nearby emergency resources.

Overall, the project brings important disaster-response functions into a single platform. It aims to support faster emergency reporting, better communication, community participation, location-based assistance, and improved coordination between affected users and response teams. Future development can further enhance the system through real-time notifications, automatic volunteer matching, AI-based disaster analysis, offline support, multilingual features, and integration with external emergency services.

12.2 What Was Achieved

The Disaster Alert & Community Response App successfully established a centralized platform concept for supporting users during disaster and emergency situations. The project achieved the development of the main application structure and user-facing modules, including user registration and login, disaster alerts, incident reporting, emergency help requests, SOS, community assistance, live disaster map, shelters, emergency services, and profile management.

The application architecture was organized using React + Vite for the frontend, FastAPI for the backend, and PostgreSQL for database management. The frontend pages and navigation were developed, form validation was implemented, and alert filtering functionality was added. The project also defined REST API endpoints for authentication, alerts, incidents, help requests, SOS, community responses, shelters, emergency services, and user profiles.

Testing activities were planned using Postman, frontend testing, and integration testing to verify communication between the application components. Overall, the project established the core structure required for a centralized disaster-response platform and provides a foundation for further backend, database, real-time notification, and location-service integration.

12.3 Skills Learned During Development

During the development of the **Disaster Alert & Community Response App**, several technical and practical skills were developed. The project provided experience in **React and Vite** for creating frontend pages, components, forms, and navigation. It also improved understanding of **REST API architecture and FastAPI** for designing backend services and communication between the frontend and backend.

The project also helped develop skills in **database design and PostgreSQL**, including planning tables for users, disaster alerts, incidents, help requests, community responses, shelters, and emergency services. Additional skills

were gained in **API testing using Postman**, frontend validation, integration testing, and debugging application issues.

The development process also provided experience with **location-based services and map integration**, particularly for incident tracking, shelters, and emergency services. Overall, the project improved skills in **full-stack application development, system architecture, database management, API development, testing, debugging, and problem-solving**.

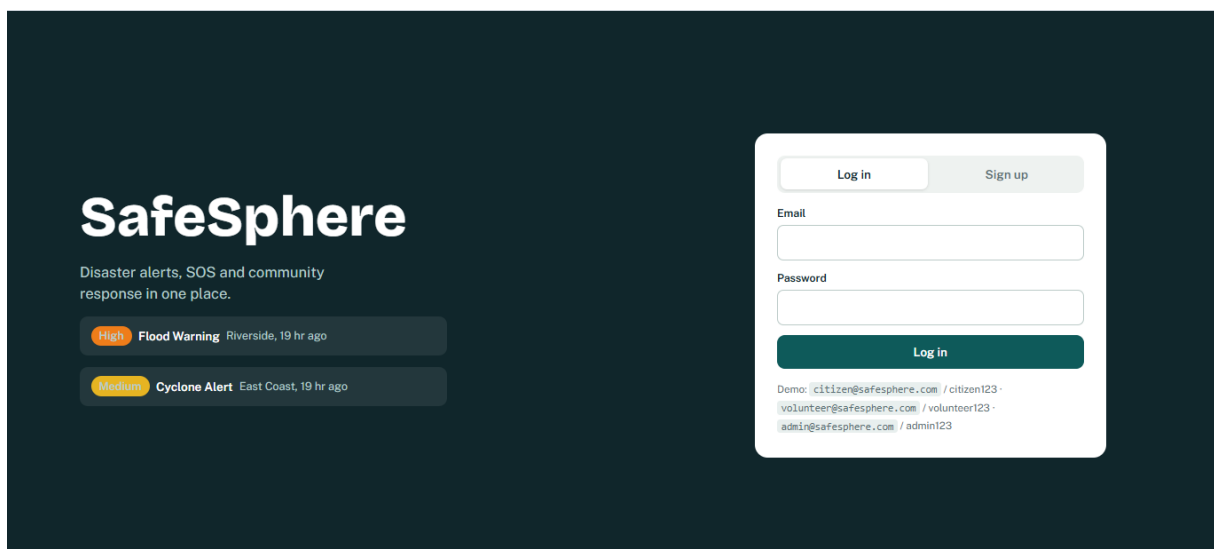
Chapter 13:REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>

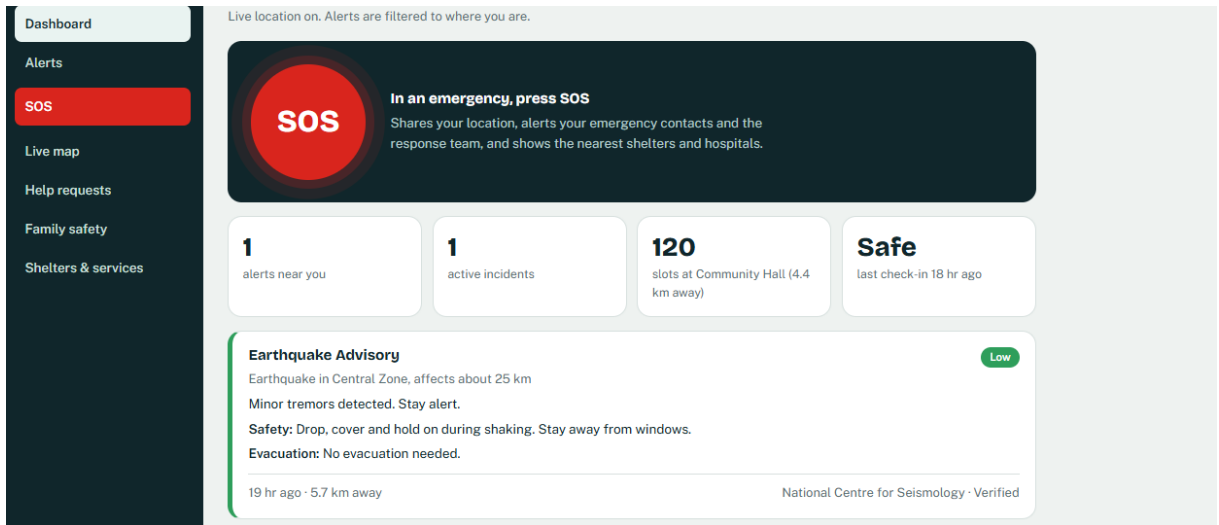
CHAPTER -14 APPENDICES

Screenshots of the app

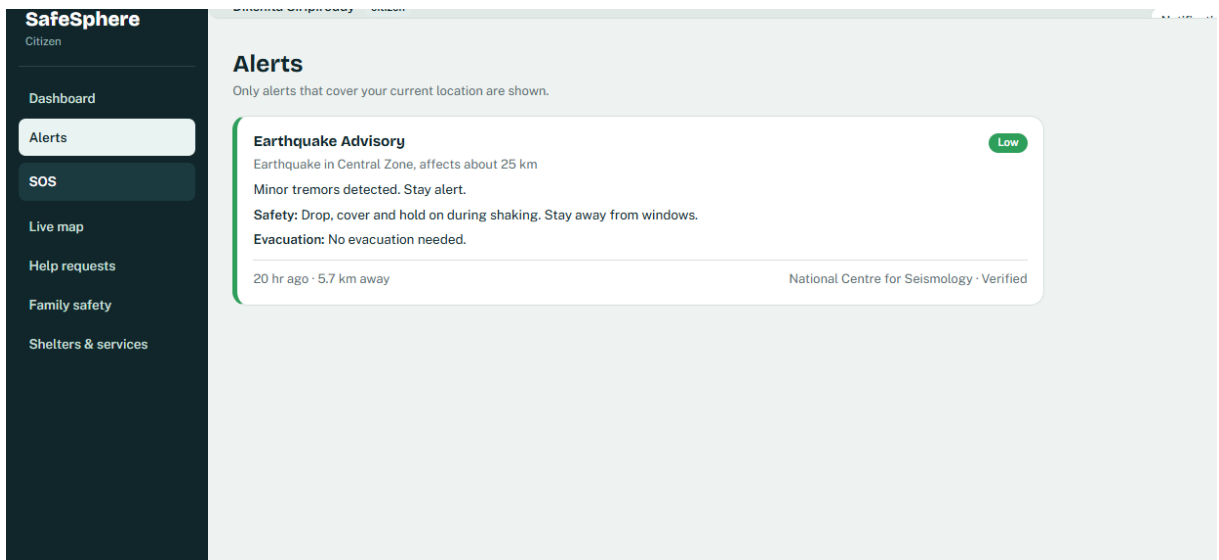
- Login Page



- Dashboard



- Alerts



- SOS

Citizen

Dashboard
Alerts
SOS
Live map
Help requests
Family safety
Shelters & services

Emergency SOS

Pick the emergency, then send. This alerts people right away, so use it for real emergencies.

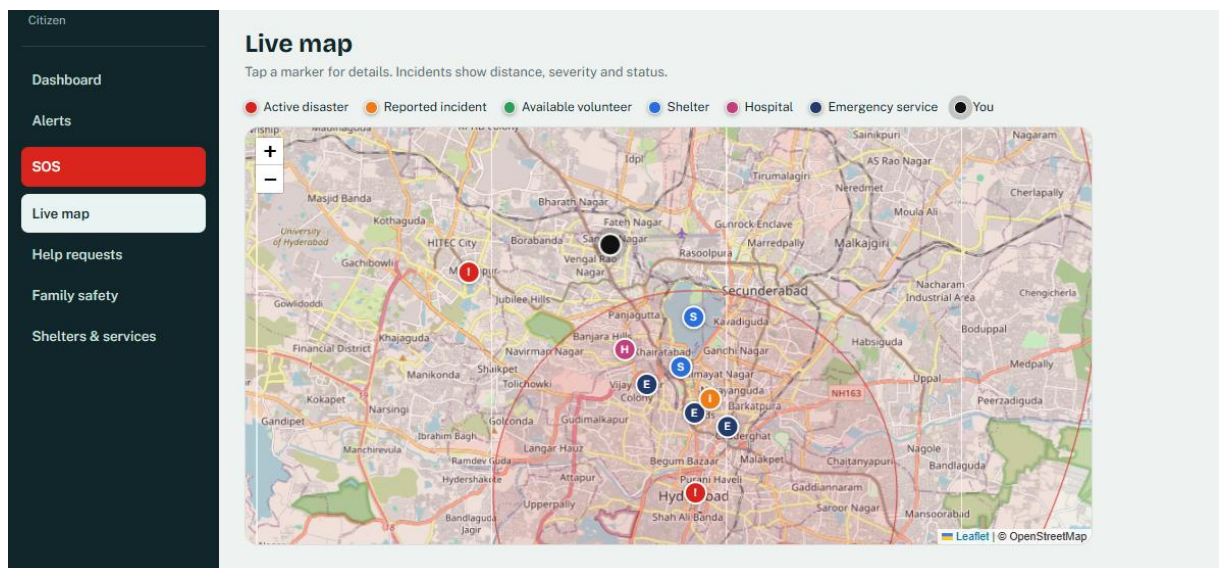
Medical emergency
Trapped
Fire
Flood
Accident
Missing person
Other

Where are you (optional)

Anything responders should know (optional)

Send SOS

- Live Map



- Help Requests

Dashboard
Alerts
SOS
Live map
Help requests
Family safety
Shelters & services

Help requests

Most urgent first.

Critical Life-threatening / trapped
High Urgent medical or evacuation assistance
Medium Food, water, shelter
Low General assistance

Medical Help
Critical

Priya · Sector 12 · 6.5 km away

Elderly relative needs urgent medical help.

Pending · 19 hr ago

Food
Medium

Arjun · Lake View · 4.4 km away

Family of five needs food supplies.

Accepted · 19 hr ago

Want to help others? Sign up as a volunteer to respond to requests.

Sample code snippets

- Backend, main.py

```
• import logging
• import os
• from contextlib import asynccontextmanager
• from pathlib import Path
•
• from dotenv import load_dotenv
• from fastapi import FastAPI
• from fastapi.middleware.cors import CORSMiddleware
•
• load_dotenv(Path(__file__).resolve().parent / ".env")
•
• import routes_auth # noqa: E402
• import routes_public # noqa: E402
• import routes_safety # noqa: E402
• import routes_user # noqa: E402
```

- routes_safety.py

```
• @router.put("/me/location", tags=["Location"])
• def update_my_location(data: LocationIn, db: Database = Depends(get_db),
• user: User = Depends(get_current_user)):
•     _save_location(db, user.user_id, data.latitude, data.longitude)
•     return {"ok": True}
•
• def _contact_users(db: Database, user: User) -> dict[str, int]:
•     """normalised phone -> user_id of every OTHER SafeSphere user."""
•     out = {}
•     for u in db.users.find({"phone": {"$ne": None}, "user_id": {"$ne":
• user.user_id}}, {"_id": 0, "user_id": 1, "phone": 1}):
•         p = norm_phone(u["phone"])
•         if p:
•             out[p] = u["user_id"]
•     return out
•
```

- App.jsx

```
• function AuthScreen({ onAuth, alerts }) {
•     const [mode, setMode] = useState("login");
•     const [err, setErr] = useState("");
•     const [busy, setBusy] = useState(false);
•     const submit = async (e) => {
•         e.preventDefault();
•         const f = new FormData(e.currentTarget);
•         setBusy(true); setErr("");
•         try {
```

```

•     onAuth(mode === "login" ? await api.login(f.get("email"),
f.get("password"))
•       : await api.register({ name: f.get("name"), email:
f.get("email"), phone: f.get("phone") || null, password:
f.get("password"), role: f.get("role") }));
•     } catch (x) { setErr(x.message); setBusy(false); }
•   };

```

Installation/setup instructions

Backend Setup

1. Open the project in **Visual Studio Code**.
2. Open **Windows PowerShell**.
3. Navigate to the backend folder: `cd backend`
4. Run the backend: `./run_backend.bat`
5. The backend will run at:

`http://127.0.0.1:8000`

Frontend Setup

1. Open the frontend folder in Visual Studio Code.
2. Open a new terminal in VS Code.
3. Navigate to the frontend directory:

`cd frontend`

4. Install the required packages:

`npm install`

5. Start the React (Vite) development server:

`npm run dev`

6. Open the URL displayed in the terminal (typically):

`http://localhost:5173`

Geo Tag photos with guide

Review forms with guide signatures